Containerlab Node Manager — Student Quick Start

Guide revision 2 · tested with manager release 1.31.6 · October 2026

This guide walks you through the manager from the browser: deploying a lab, working on its devices from the command line, and saving your lab to Git so your work is not lost even after the lab itself is destroyed. Pick the path that matches what you were given and follow it step by step; each step says what to do and what you should see.

Your starting point	Follow
I was given a lab by my instructor	Scenario A
I want to build my own lab and keep it in Git	Scenario B

Before you start 1
Scenario A: use a lab your instructor gave you 3
Scenario B: build your own lab and keep it in Git 14
Reference 25

Before you start

What you have been given

Item	What it is
Manager address	`http://192.168.132.134:8081` — there is no sign-in
Your GitHub account	The one the lab VM is signed in to: `ArchRuger` here, your own for you
A terminal on the lab VM	A shell as your VM account (`archtop` here), needed only in step B9
Instructor materials	The `link-basics` lab on the lab VM, with its saved states in the instructor's course repository `https://github.com/pruger-dev/netlab-course` (read-only for you; Scenario A)
Your own repository	Scenario A: your private copy of the course repository, made with **Use this template › Create a new repository** (here `campaign2-netlab-course-student`). Scenario B: one you create in step B5

Everything below happens in one of three places: **your computer** (the browser you are reading this in, and any terminal you run on your own machine), **the lab VM** (the server that actually runs the lab's devices, and where the manager itself lives), and **your online repository** (your account on

GitHub, out on the internet). A file can exist in one of these and not the others until you deliberately move it — keeping track of which place you are looking at is most of what this guide teaches.

A few Git words come up along the way. A **repository** is a project's saved history, kept on GitHub and also as a copy elsewhere; a **commit** is one saved snapshot of some files with a short message; to **push** — this guide calls it **upload** — is to send your commits to the copy on GitHub; and to **clone** is to copy a whole repository from GitHub onto another machine. Each word is used again later without repeating its definition.

Two buttons in every lab's header do most of the work. **Save** puts the whole lab as it is now — the topology file, the map and the configuration of every device — into one folder of your repository, and **Load** puts a saved state back onto the running devices. Between them sits a small status button, the *save chip*, that always says where the lab stands (**Not saved yet**, **1 save to upload**, **Saved 3 min ago**, **Running Start**, …). Nothing reaches GitHub until you press **Upload**.

Scenario A: use a lab your instructor gave you

In this scenario you deploy a lab your instructor already prepared, look at it from the command line, load the instructor's starting state, make a small change, save your work to Git, and practice recovering from a fault — all using `link-basics`, two routers, `r1` and `r2`, joined by one link.

> **A save is the whole lab: the topology file, the map and, for each device, two files — the readable configuration (`.cfg`) and the copy Load uses (`.eoscfg`) — plus a `manifest.json` that lists them.** Your instructor's `link-basics` topology is staged on the lab VM and your first save copies it into your own repository, so you never save it separately in this scenario.

A1 Open the manager

Action In a browser on your computer, go to `http://192.168.132.134:8081`.

Expected result Home shows a **Deploy** box, a **Build** box, a **Manager ▾** menu and, below them, your labs. On a new manager the list reads **No labs yet**; the figure shows only the two boxes.

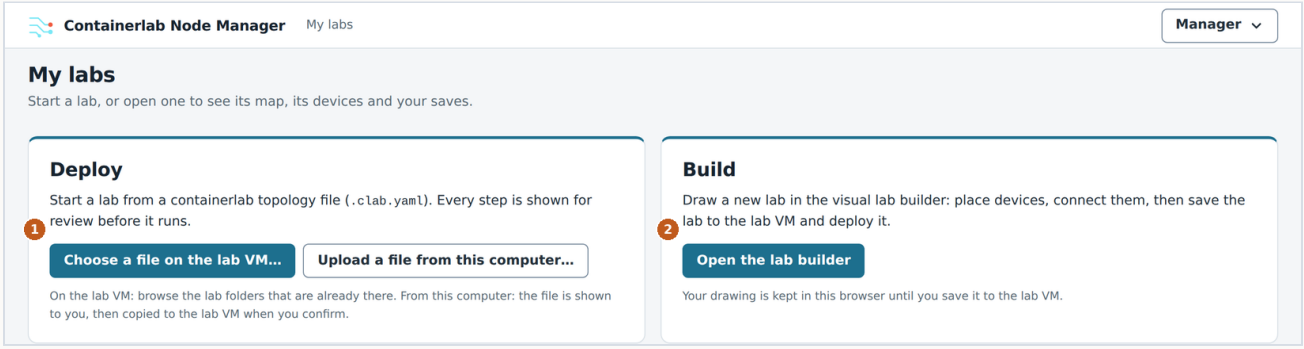


Figure A.1 — Home: the Deploy and Build boxes.

A2 Deploy the lab

Action In the Deploy box, click **Choose a file on the lab VM…**. The **Deploy a new lab** dialog opens on the lab folders on the VM; open `/srv/containerlab-node-manager/projects`, then the `link-basics` folder inside it, and click ◇ **link-basics.clab.yml**.

Expected result A **Topology file** dialog opens showing the **File location on the VM** and the topology (YAML), with buttons **Preview topology**, **Edit topology…**, **Add to My labs without starting** and **Deploy lab**.

Action Click **Deploy lab**.

Expected result A review titled **Start link-basics?** opens: "Creates and starts the devices in this topology as its file describes them, including any hooks, mounts and image pulls. Nothing is deleted; device logins open as the devices boot." It also shows the exact command that will run on the VM. Confirm with **Start lab**.

If not If `link-basics` does not appear in the folder browser, ask your instructor to confirm it is staged on the lab VM — the browser only shows lab folders the manager trusts. If the page says

"Wait for the current lab operation to finish.", someone else's operation is running on the manager; wait a moment and click again.

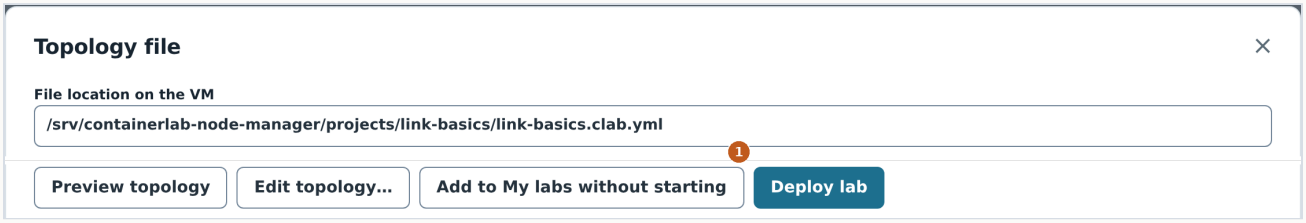


Figure A.2a — The Topology file dialog for `link-basics.clab.yml`.

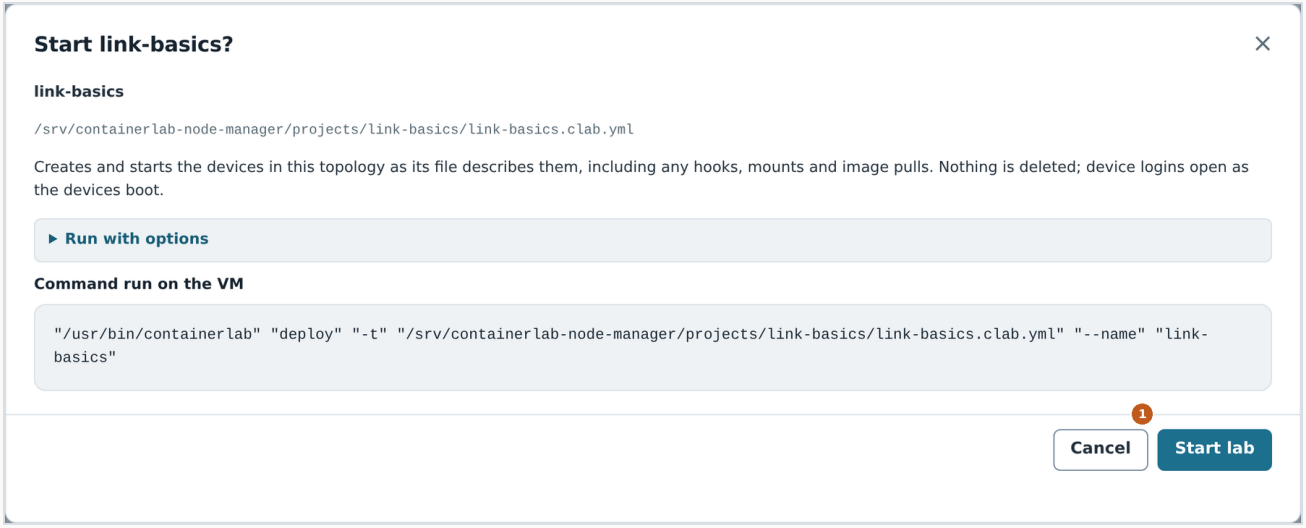


Figure A.2b — The Start link-basics? review.

A3 Wait for the devices to become ready

Action Stay on the lab page while `r1` and `r2` start.

Expected result A running container is not yet a router that accepts a login. Right after you confirm, the header reads **Starting lab** and **0 of 2 devices ready**, and each device's row reads **Starting** until the manager can log in and get an answer from it, then **Ready** — about a minute on this VM; up to a few minutes on a slower one. Wait until the header reads **Running** and **2 of 2 devices ready**; on the **Devices** tab both rows read **Ready**.

If not A pill reading **Needs credentials** means the device is up but the manager could not log in — check the login with your instructor. **Needs attention** after several failed attempts offers **Test login now** to retry once the problem is fixed.

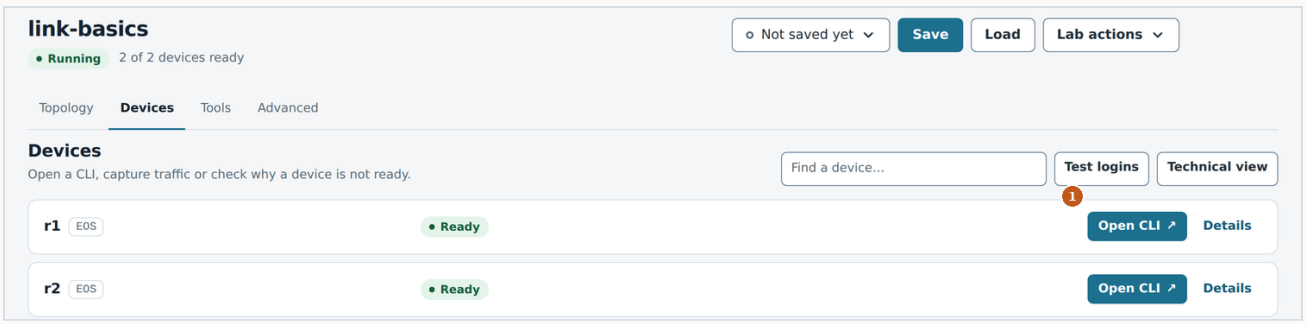


Figure A.3 — Both devices read Ready.

A4 Look around from the command line

Action On the **Devices** tab, click **Open CLI ↗** next to `r1`. It opens a terminal in a new browser tab; wait until its status reads **Connected**, then at the `r1>` prompt run:

```
r1>show version
r1>show interfaces status
r1>show ip interface brief
```

Expected result `show version` reports the platform; `show interfaces status` shows `Et1` as `connected`, since the link between `r1` and `r2` is physically up; `show ip interface brief` lists only `Management0` — `Ethernet1` has no address yet, so EOS does not list it there. That is expected: the lab starts with the link wired but nothing configured on it.

```
r1>show interfaces status
Port       Name   Status       Vlan     Duplex Speed  Type            Flags Encapsulation
Et1               connected    1        full   1G     EbraTestPhyPort
Ma0               connected    routed   a-full a-1G   10/100/1000

r1>show ip interface brief
                                                                              Address
Interface         IP Address            Status       Protocol           MTU    Owner
----------------- --------------------- ------------ -------------- ---------- -------
Management0       172.20.20.11/24       up           up                 1500

r1>
```

Figure A.4 — `r1`'s terminal after the show commands.

A5 Connect your repository and make the first save

Action Make sure you already have your own copy of the course repository — made on GitHub with **Use this template › Create a new repository** before you begin. Open the lab's **Topology** tab. The header shows the save chip, **Not saved yet**, beside **Save** and **Load**. Click the chip: its panel offers a place to save — "Your first save goes to …, in a folder named link-basics" — with **Save**, **Choose another place** and **Connect by URL…**. (On a VM that knows no repository yet, the panel asks for the repository's address directly.) Click **Connect by URL…**. In the drawer **Where should link-basics save?**, paste `https://github.com/ArchRuger/campaign2-netlab-course-student.git` (your own copy's address, not the instructor's) into **Repository address (HTTPS)**, set **Folder** to `link-basics/work`, and click **Save here**. You are never asked for a password or a token: the lab VM uses its own GitHub login.

Expected result The manager connects the repository and makes the lab's first save at once — about ten seconds. The chip reads **1 save to upload** and its panel opens, titled **Not uploaded yet**: "This is the first save here: 2 devices, the topology and the map." with "To: campaign2-netlab-course-student › link-basics/work" and the buttons **Upload**, **Not now**, **See changes** and **Details**. **See changes** lists exactly what the save holds: `Topology file link-basics.clab.yml added`, `Map link-basics.clab.yml.annotations.json added`, `r1 r1.cfg added` and `r2 r2.cfg added`. Click **Upload**.

Expected result The chip reads **Saved just now**. On GitHub, `link-basics/work/latest/` now holds seven files: the topology, the map, `r1.cfg`, `r1.eoscfg`, `r2.cfg`, `r2.eoscfg` and `manifest.json`.

If not If the manager cannot reach the repository, check that you made your own copy of the course repository first — the address must point at your copy, not the instructor's. If the chip

panel still says "Your first save goes to ..." after **Save here**, another operation was running when you clicked; press **Save** once it has finished.

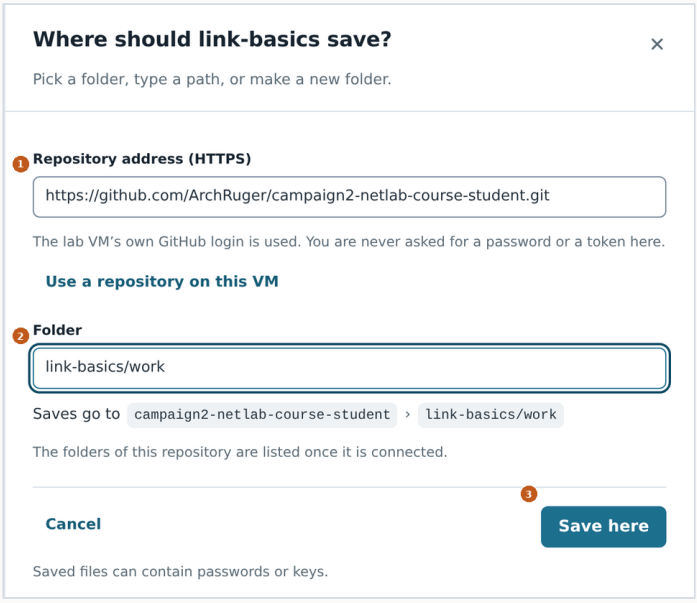


Figure A.5a — The drawer Where should link-basics save?, filled in.

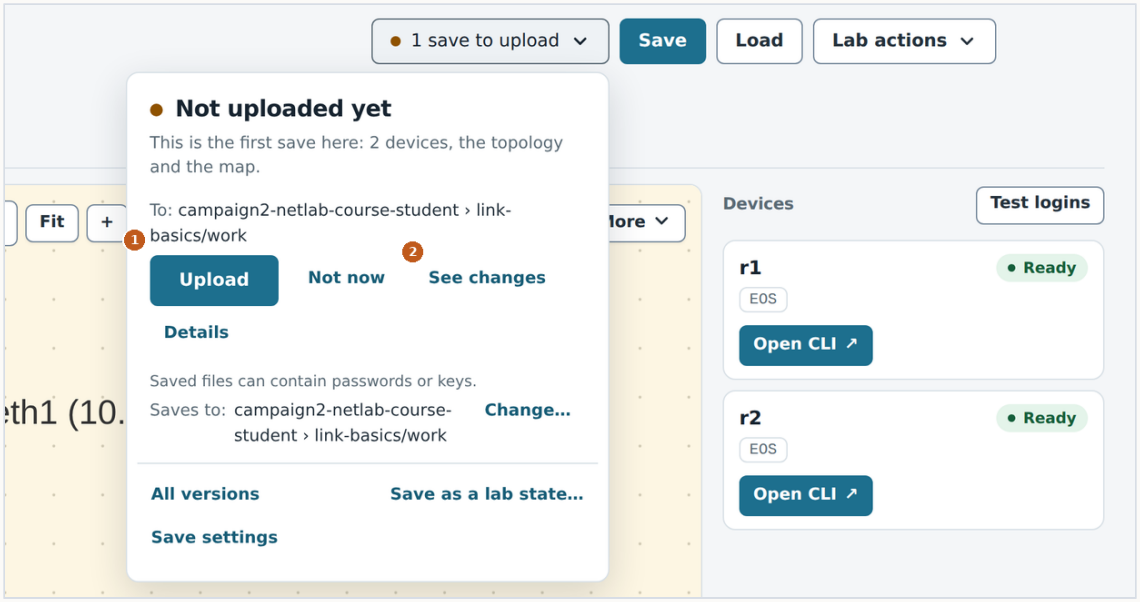


Figure A.5b — The first save, waiting for Upload.

A6 Load the starting state

Action Click **Load** in the lab header. The panel lists **Your saves** (your **First save**) and, under **Lab states**, the three states your instructor put in the repository: **Broken-01**, **Solution** and **Start**, each covering 2 devices. Click **Start**.

Expected result The manager checks the state against your devices and shows **Load Start?**: "The running configuration of the ticked devices is replaced. The current one is backed up first; nothing reboots." Each device is listed with a tick and how far it is from the state — "r1 EOS · 6 lines differ", "r2 EOS · 6 lines differ". Click the red **Load** — if a device cannot be reached again within five minutes, it undoes the change on its own.

Expected result Within about ten seconds the chip reads **Running Start** and a message says "Start loaded on 2 devices." Its panel reads "Loaded just now on all 2 devices." with **Undo this load**. Go back to `r1`'s terminal tab from A4 (still open) and run `ping 10.0.0.2` — it succeeds, since `Start` addresses the link `10.0.0.1/30` on `r1` and `10.0.0.2/30` on `r2`.

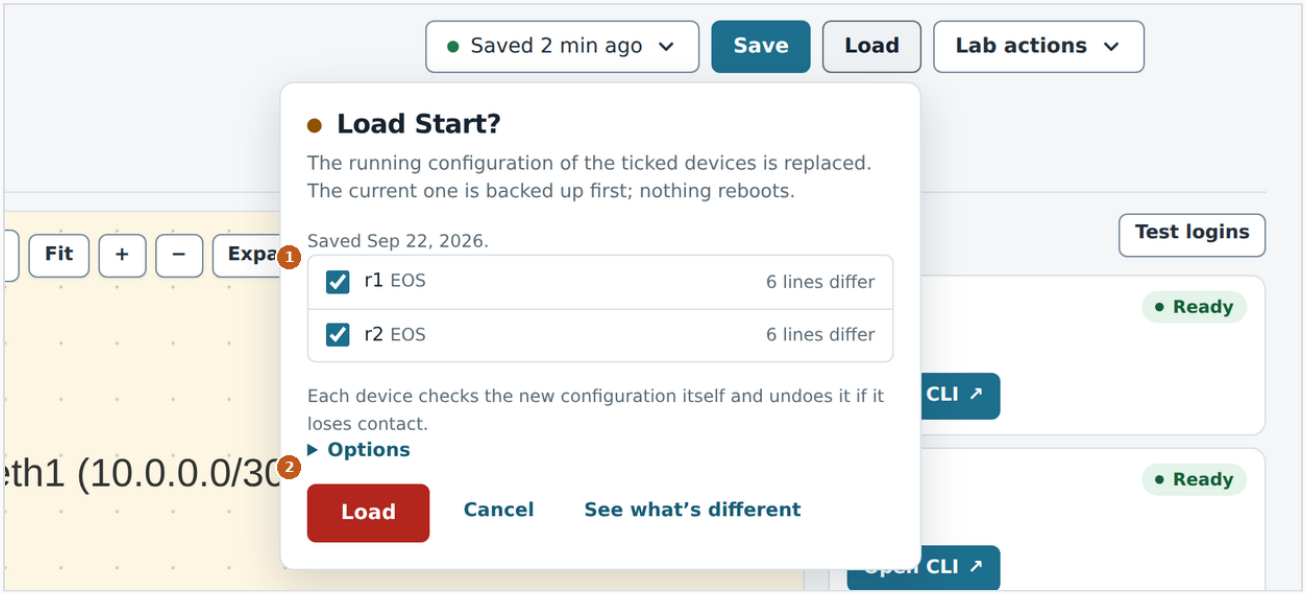


Figure A.6a — Load Start?, listing every device before anything changes.

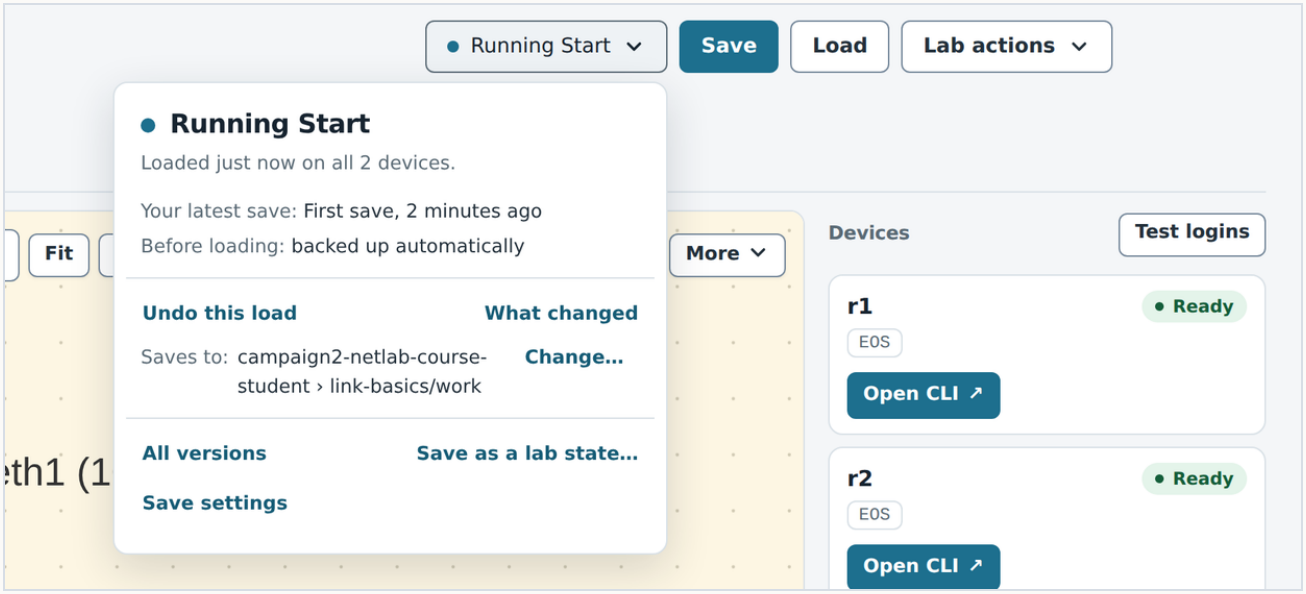


Figure A.6b — Running Start.

A7 Make a change on r1

Action On `r1`'s CLI:

```
r1>enable
r1#configure terminal
r1(config)#interface Loopback0
r1(config-if-Lo0)#description r1 loopback
r1(config-if-Lo0)#ip address 10.255.0.1/32
r1(config-if-Lo0)#end
r1#write memory
r1#show ip interface brief
```

On EOS a configuration command is active the moment you enter it; `write memory` only keeps it across a device restart, and **Save** in the next step captures the running configuration either way.

Expected result `show ip interface brief` now also lists `Loopback0` with address `10.255.0.1/32`. `r2` is untouched for now — its own loopback comes in step A9.

```
Last login: Tue Oct 27 03:34:25 2026 from 172.20.20.1
r1>enable
r1#configure terminal
r1(config)#interface Loopback0
r1(config-if-Lo0)#description r1 loopback
r1(config-if-Lo0)#ip address 10.255.0.1/32
r1(config-if-Lo0)#end
r1#write memory
Copy completed successfully.
r1#show ip interface brief
                                                                                Address
Interface         IP Address            Status       Protocol            MTU    Owner
----------------- --------------------- ------------ -------------- ----------- -------
Ethernet1         10.0.0.1/30           up           up                 1500
Loopback0         10.255.0.1/32         up           up                65535
Management0       172.20.20.11/24       up           up                 1500

r1#
```

Figure A.7 — `Loopback0` added and read back on `r1`.

A8 Save your work

Action Click **Save** in the lab header.

Expected result The manager reads both devices and writes the lab into your folder as one save, then says in one sentence what changed: the chip reads **1 save to upload** and its panel, **Not uploaded yet**, says "2 devices changed since your last save: r1 and r2. 12 lines added, 4 removed." — both routers changed because `Start` addressed their link and `r1` also has its loopback. Click **See changes** for the line-by-line view, device by device; the **What changed** drawer has its own **Upload**. Click **Upload**. **Not now** would leave the save waiting on the lab VM instead (the chip would keep reading **1 save to upload**).

Expected result The chip reads **Saved just now**. On GitHub, `link-basics/work/latest/` holds the same seven files with `r1.cfg`, `r1.eoscfg`, `r2.cfg` and `r2.eoscfg` updated — a save replaces `latest` in place; Git keeps the earlier versions.

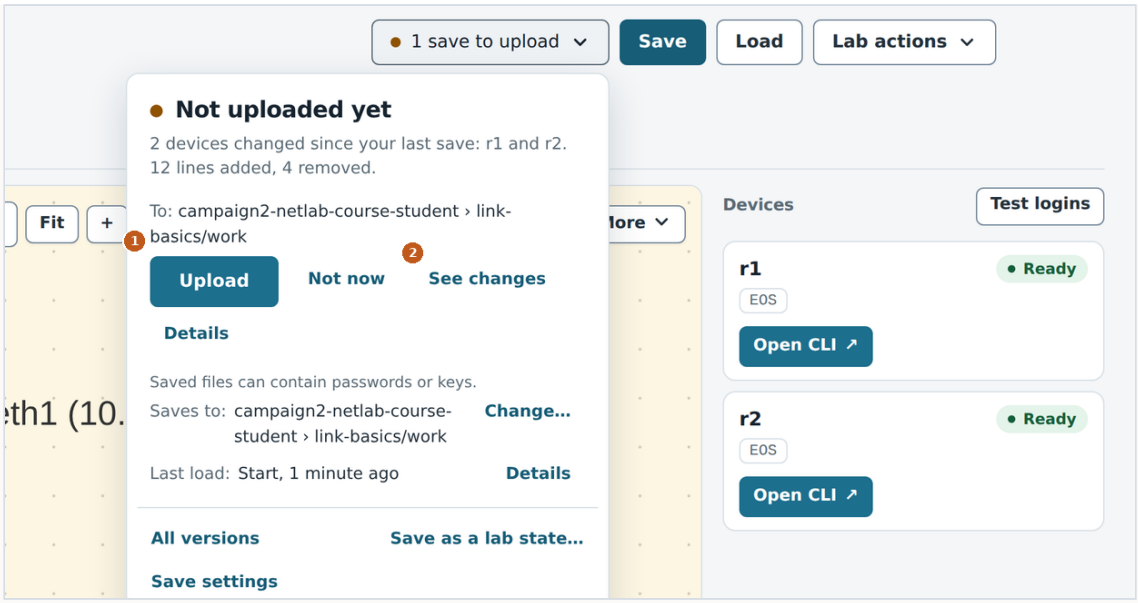


Figure A.8a — The sentence after Save, with Upload, Not now and See changes.

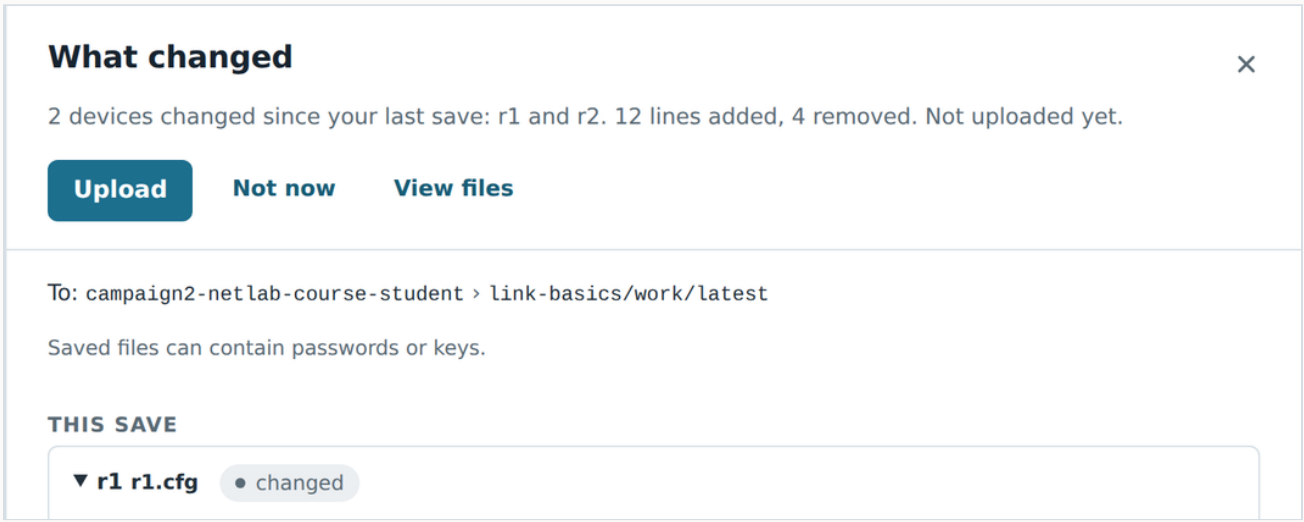


Figure A.8b — The top of the What changed drawer.

A9 Checkpoint, then add r2's loopback

Action Click the chip. Under **Saved just now** its panel shows the save's name (the manager named it "r1 and r2 changed") in a field and a box **Keep as a checkpoint**. Type `loopback-added` into the field and press Enter, tick **Keep as a checkpoint**, then click **Upload** when the panel says "Checkpoint loopback-added kept. It is not uploaded yet."

Action Now give `r2` its own loopback: on `r2`'s CLI, run the same commands as in A7, using `description r2 loopback` and `ip address 10.255.0.2/32`, then `write memory`. Click **Save** again — the sentence reads "1 device changed since your last save: r2. 4 lines added, 0 removed." — and **Upload** that save too.

Expected result Click the chip, then **All versions**. The drawer lists **Your saves** (**r2 changed**, **loopback-added**, **First save**), **Checkpoints** (`loopback-added`) and the instructor's **Lab states**. On GitHub, `link-basics/work/latest/r2.cfg` now contains `Loopback0`, while `link-basics/work/checkpoints/loopback-added/r2.cfg` does not — the checkpoint captured the lab before `r2` had its loopback. There is no nested `latest/latest`.

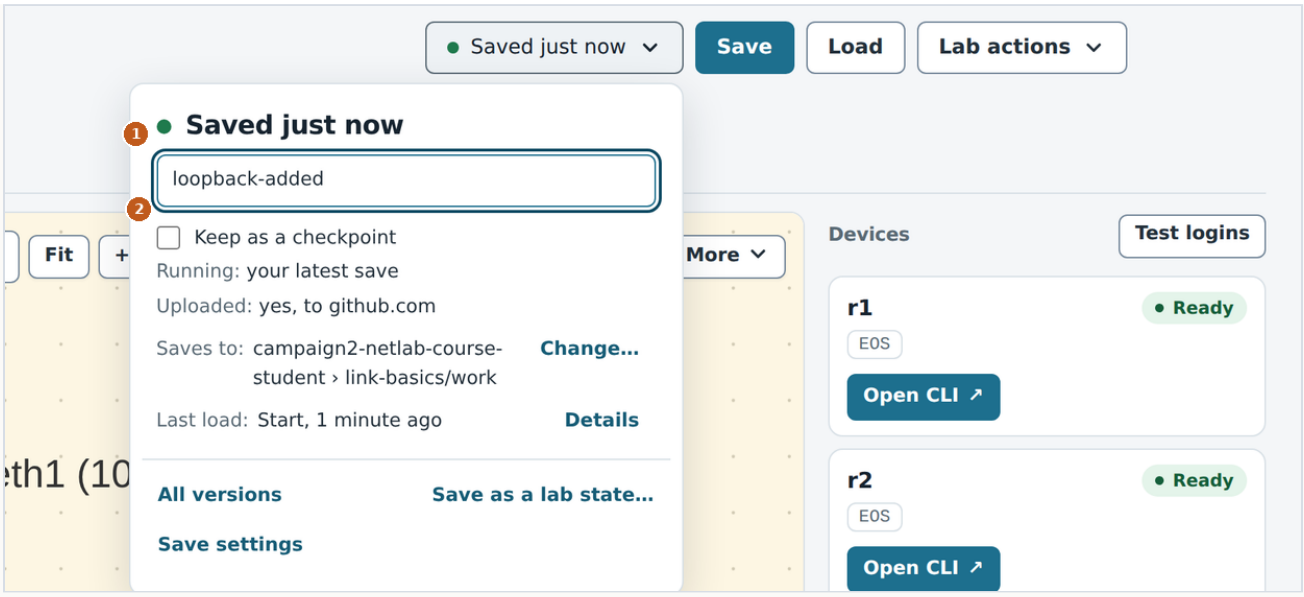


*Figure A.9a — Naming the save `loopback-added`; the box **Keep as a checkpoint** is under the name.*

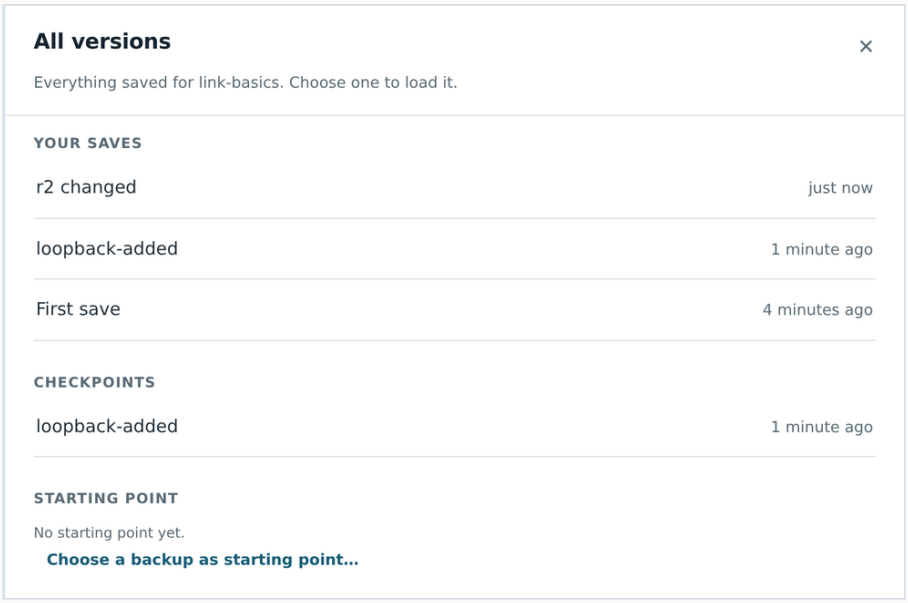


Figure A.9b — All versions: your saves, the checkpoint and the lab states.

A10 Break it, then recover

Action Click **Load**, click **Broken-01**, and click the red **Load**.

Expected result Both devices are loaded; the chip reads **Running Broken-01**. On the **Topology** tab the link turns into a dotted red line; on `r1`'s CLI the ping fails and `show interfaces status` shows `Et1` as `notconnect` — the instructor's broken state shut down `r2`'s `Ethernet1`. See if you can spot why before you move on.

Action The quickest way back: click the chip and **Undo this load**. The confirmation reads "Undo loading Broken-01?"; click the red **Load**.

Expected result The chip reads "Running the configuration from before Broken-01", and `ping 10.0.0.2` from `r1` succeeds again. Undo is a load of the backup the manager made just before the load, so it is offered for as long as that backup is kept.

Action The way that always works is to load your own saved work. Load **Broken-01** once more, then click **Load** and your newest save — the first row under **Your saves**, "r2 changed" — and the red **Load**.

Expected result The chip reads **Running r2 changed**, `ping 10.0.0.2` succeeds and both loopbacks are back. Loading never changes where the lab saves: the chip panel still says "Saves to: campaign2-netlab-course-student › link-basics/work". Curious what changed since your checkpoint? Click the chip, **All versions**, the `loopback-added` checkpoint and **See what's different**: the drawer shows `r2.cfg` with 4 lines only in your latest save.

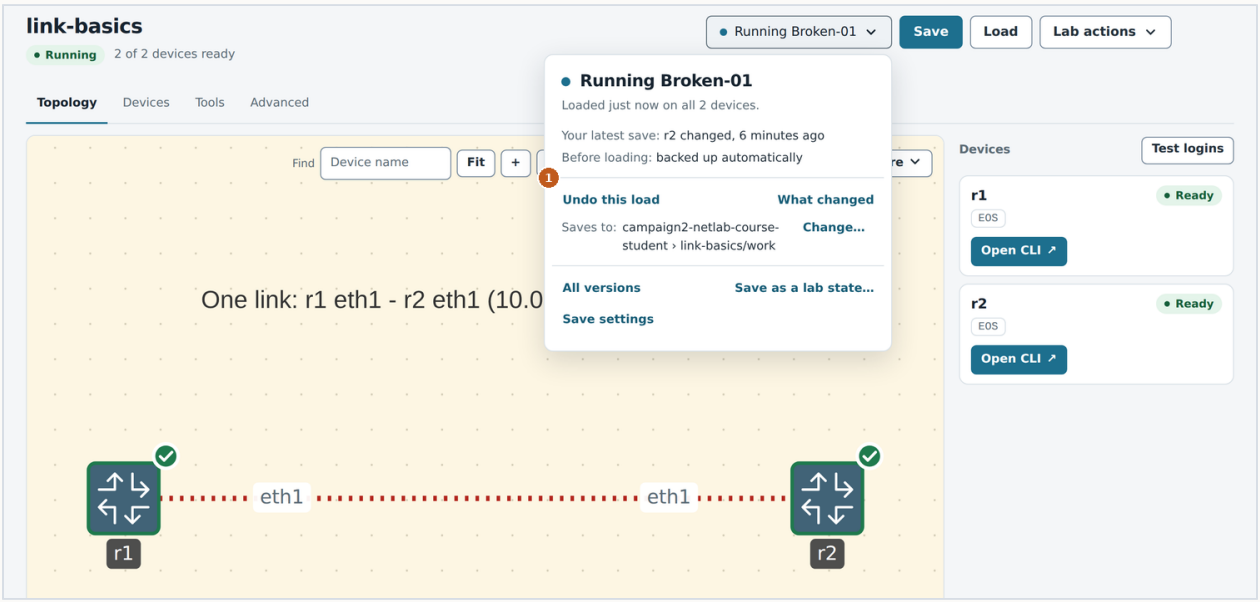


Figure A.10a — Running Broken-01: the link is down, and Undo this load is offered.

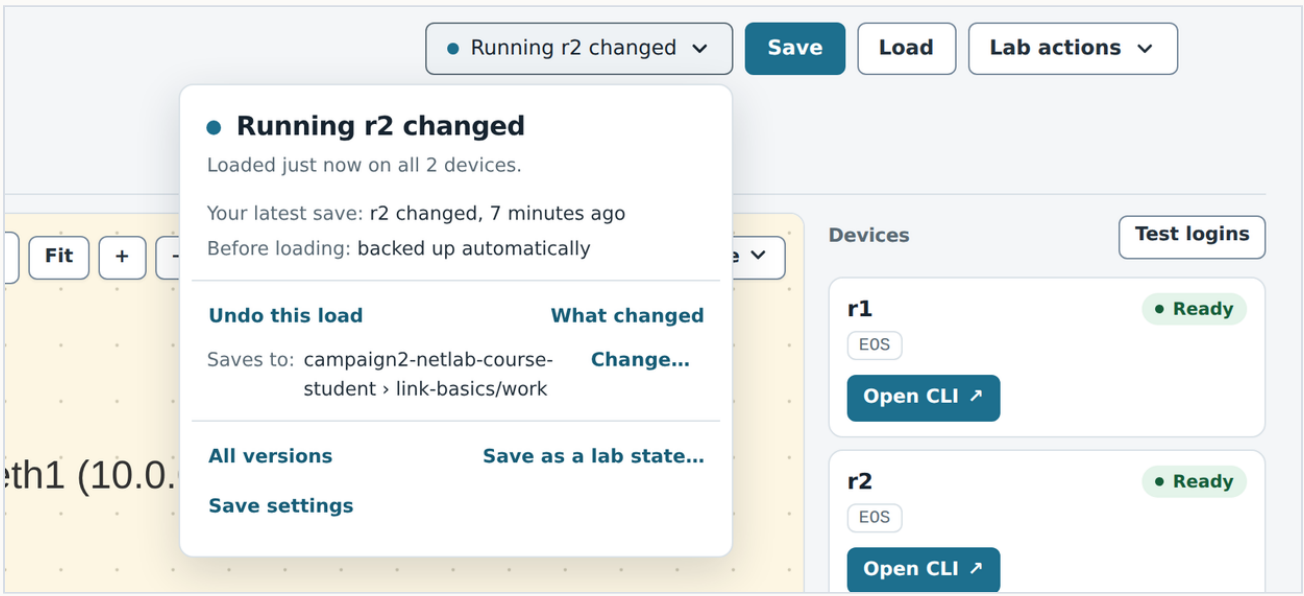


Figure A.10b — Your own newest save loaded again.

A11 Next time you open this lab

Action Close the browser, then open the manager again at `http://192.168.132.134:8081`.

Expected result The `link-basics` card on Home reads "link-basics · Running · 2 of 2 devices ready · Running r2 changed · Deployed N minutes ago", with an **Open lab** button. Open it — the chip and **Load** list your saves and the checkpoint exactly as you left them.

Action Open **Lab actions ▾** to see everything you can do with this lab — among the everyday actions are **Stop devices**, **Redeploy lab…**, **Destroy lab…** and **Remove from this manager…**; **Advanced options** unfolds **Edit map**, **Edit topology…** and more.

Open **Destroy lab…** to see its review — "The running devices are removed from the VM. Configuration changes you have not saved are lost. Your saves, lab states and backups remain." with "Last saved N minutes ago." and a **Save first** option — then click **Cancel**; there is no need to destroy this lab now. **Remove from this manager…** is even less drastic: "Nothing on the lab VM changes: the running devices and the topology files stay. What you saved stays in its repository."

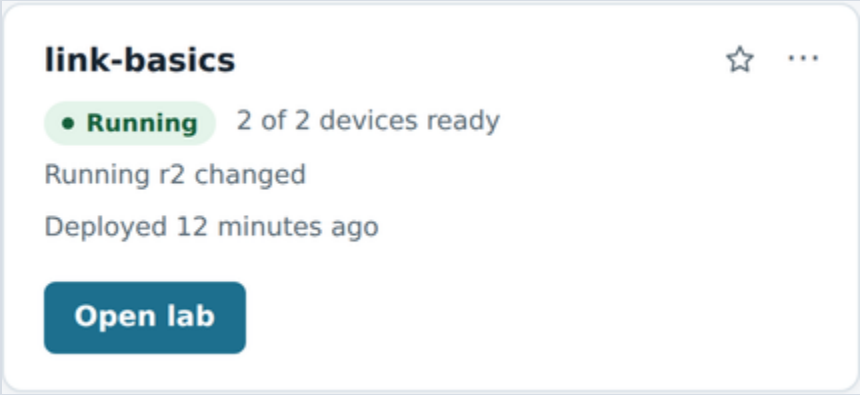


Figure A.11a — The link-basics card on Home.

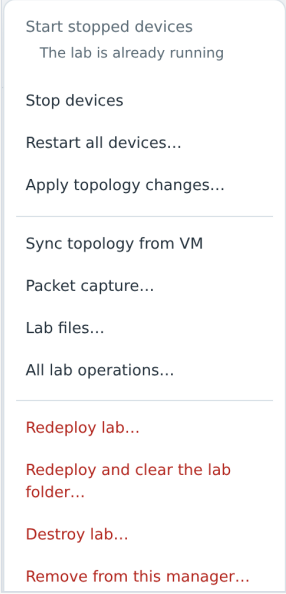


Figure A.11b — The Lab actions ▾ menu.

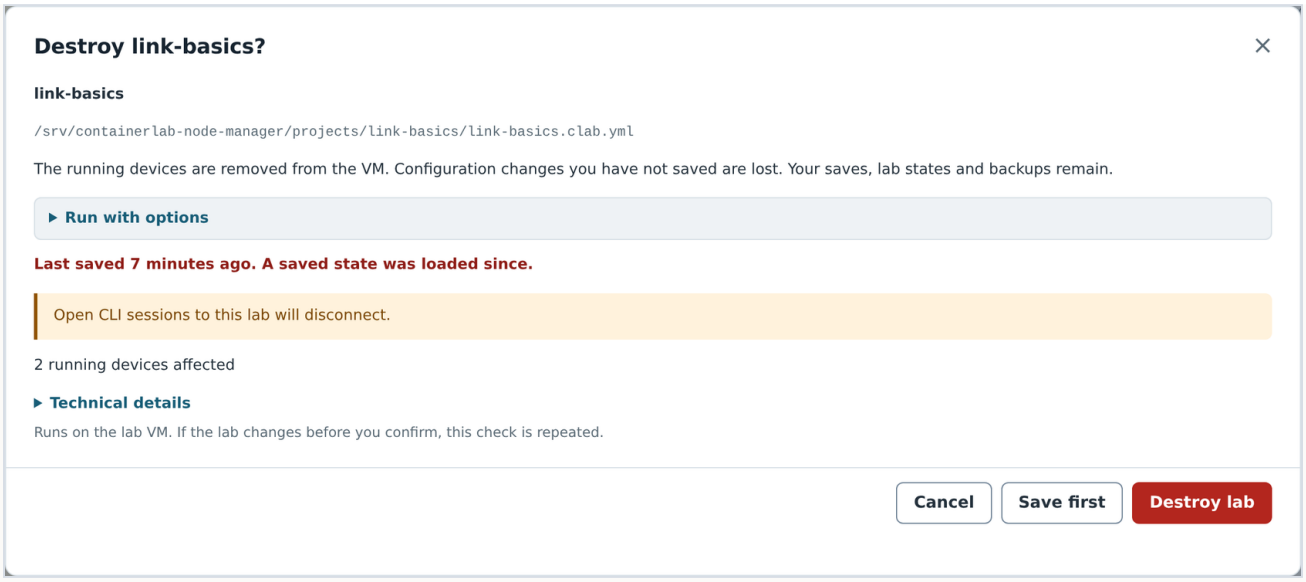


Figure A.11c — The Destroy link-basics? review, before Cancel.

Checklist

- [ ] `link-basics` deployed and both devices read **Ready**
- [ ] Repository connected, saving to `link-basics/work`; the first save uploaded
- [ ] **Start** loaded and the link pings
- [ ] A loopback added on `r1`, saved and uploaded, then kept as the checkpoint `loopback-added`
- [ ] A loopback added on `r2`, saved and uploaded (the chip reads **Saved**)
- [ ] **Broken-01** loaded, then undone, then loaded again and recovered with your own newest save

Scenario B: build your own lab and keep it in Git

In this scenario you draw your own two-router lab in the visual builder, deploy it, configure it, and set up your own repository so your work survives even after the lab is destroyed and removed from the manager. The devices are `r1` and `r2`, joined by one link, both Arista cEOS.

> **A save is the whole lab: the topology file, the map and, for each device, two files — the readable configuration (`.cfg`) and the copy Load uses (`.eoscfg`) — plus a `manifest.json` that lists them.** So one **Save** puts the lab definition and its device configurations into Git together; nothing has to be copied by hand any more.

B1 Start a new lab in the builder

Action From Home, click **Open the lab builder** in the Build box. The builder page opens in the same tab and shows **Build a lab** with **New lab…** and **Open a draft…**; click **New lab…**. Fill in **Lab name** (`my-first-lab` — letters, digits, dot, dash and underscore only) and **Lab folder on the VM** (`/srv/containerlab-node-manager/projects`). The dialog says "The lab starts as a blank canvas; drag devices from the palette to build it." Click **Create draft**.

Expected result The editor opens with an empty canvas and the **Palette** on the right (its **Nodes** tab lists the **Node Templates**, **Arista cEOS** first and starred). The page says "Drag a device from the palette on the right onto the canvas to begin. Right-click a device to link it, edit it or delete it." and the status pill reads **Draft · kept in this browser only · not on the VM yet**.

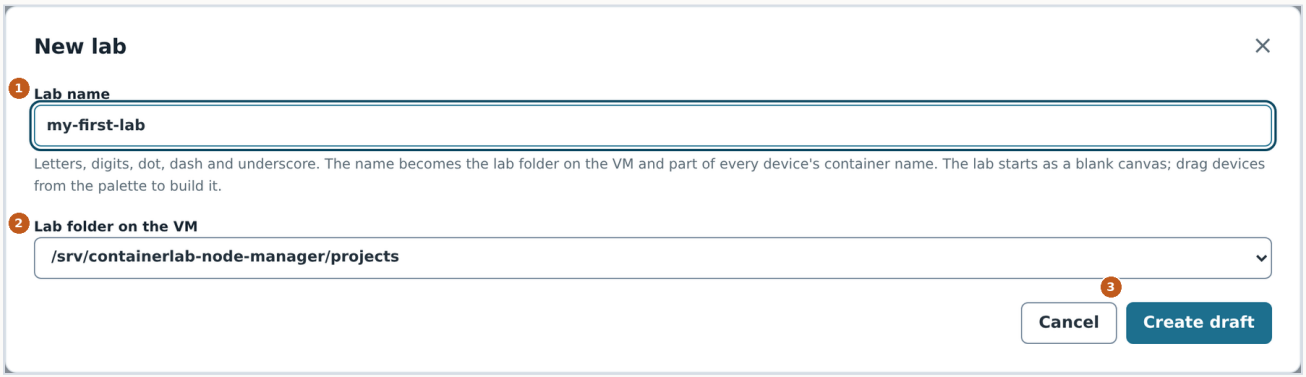


Figure B.1a — The New lab dialog, filled in.

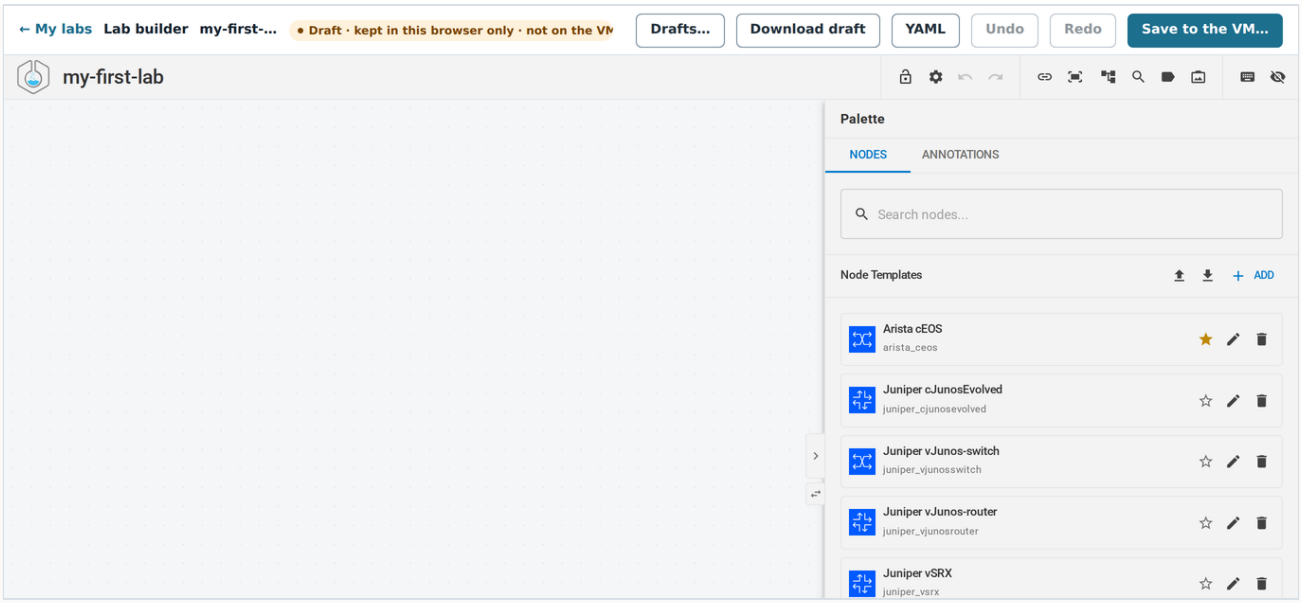


Figure B.1b — The blank canvas and the palette.

B2 Draw and check the topology

Action Drag **Arista cEOS** from the palette onto the canvas twice and put the two devices side by side. Right-click the first device, choose **Create Link**, then click the second device: a link `eth1` – `eth1` appears. Right-click the first device and choose **Edit Node**. In the Node Editor panel (tabs Basic / Configuration / Runtime / Network / Advanced) set **Node Name** to `r1`; the Basic tab already shows **Image** `n24l/ceos` and **Version** `4.35.0F`, the image this lab VM uses; on the Network tab set **Management IPv4** to `172.20.20.21`; go back to the Basic tab and click **Apply**. Repeat for the second device: name it `r2`, Management IPv4 `172.20.20.22`. A saved management address like this one is recommended because it applies again after a redeploy. While you are there, right-click empty canvas and choose **Add Text** to note what the lab is, for example "My first lab: r1 eth1 - r2 eth1".

Expected result A line under the bar reads "Image checked: available on the VM or pullable from its registry." **YAML** in the bar opens the topology as text: `name: my-first-lab`, both nodes with `kind: arista_ceos`, `image: n24l/ceos:4.35.0F` and their management addresses, and one link, `r1:eth1` – `r2:eth1`; under it the panel reads "No problems found".

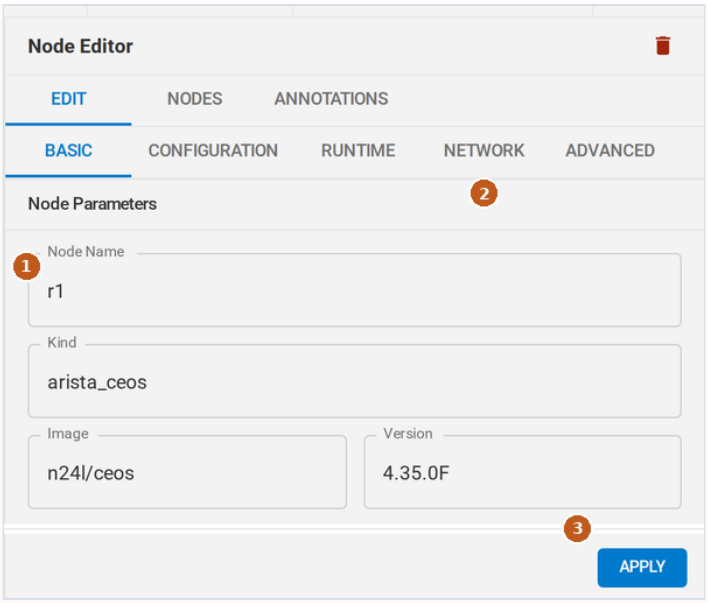


Figure B.2a — The Node Editor, naming r1.

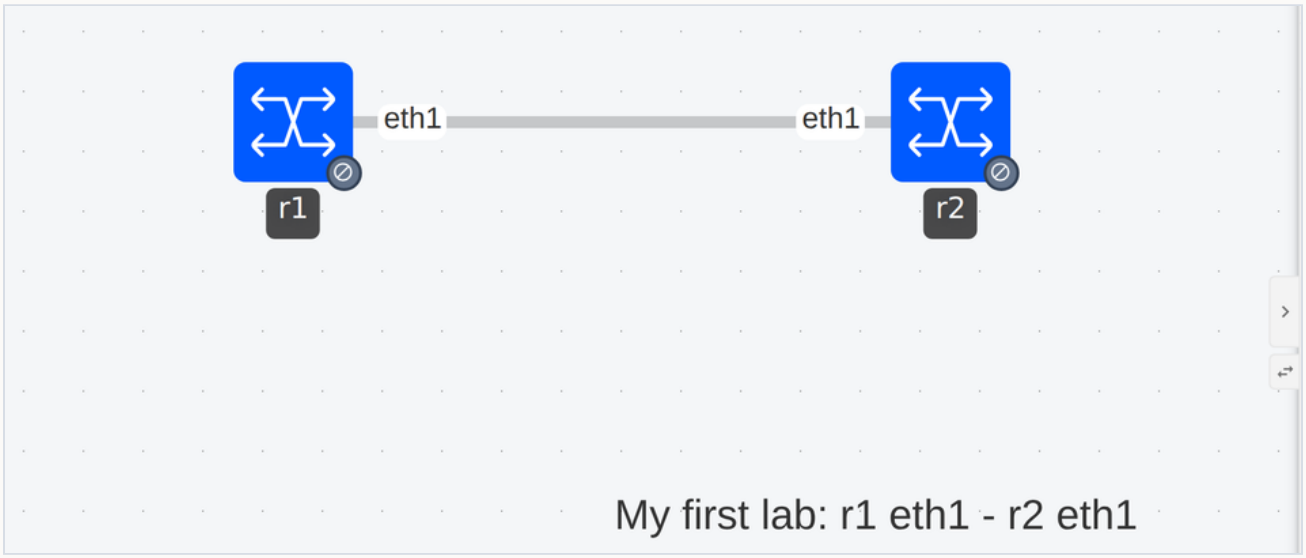


Figure B.2b — Finished canvas: r1 and r2 joined by one link, with the note.

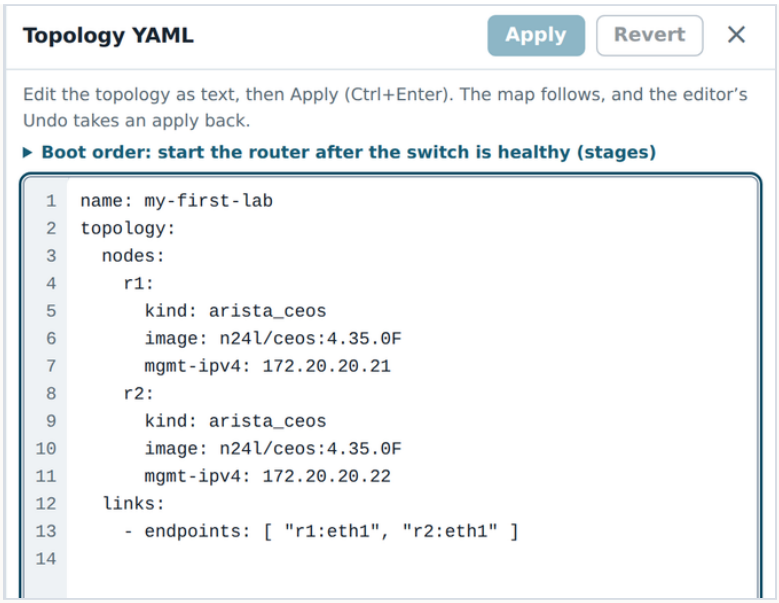


Figure B.2c — The YAML panel: the topology the drawing describes.

B3 Save to the VM and deploy

Action Click **Save to the VM…**. A review titled **Save my-first-lab to the VM?** shows the lab folder, `/srv/containerlab-node-manager/projects/my-first-lab/my-first-lab.clab.yml`, and says "Creates the lab folder … on the VM with the topology file and the map layout. Nothing on the VM is overwritten and no devices are started until you deploy it."; confirm with **Save lab**.

Expected result The result reads "✔ Save lab to the VM succeeded" with two buttons, **Add to My labs without starting** and **Deploy now**, and the status pill changes to **Saved on the VM** (the bar's button now reads **Save changes to the VM…**).

Action Click **Deploy now**. A review titled **Start my-first-lab?** opens, the same review as in Scenario A; confirm with **Start lab**.

Expected result The builder page itself does not navigate anywhere next — click **← My labs** at the top left and open the `my-first-lab` card from Home to continue.

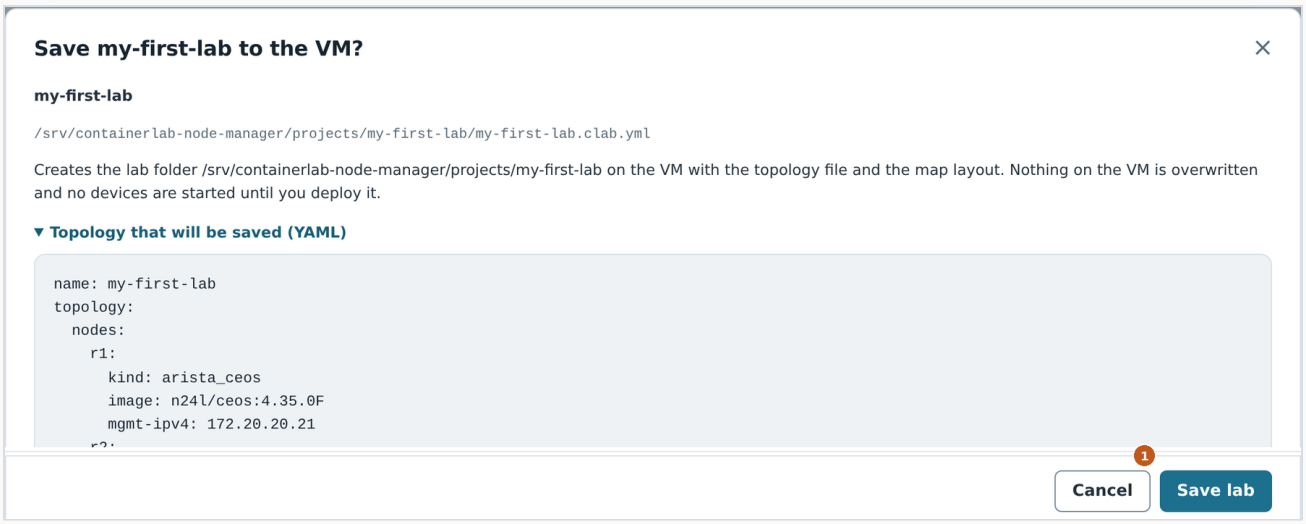


Figure B.3a — The review before the lab folder is written to the VM.

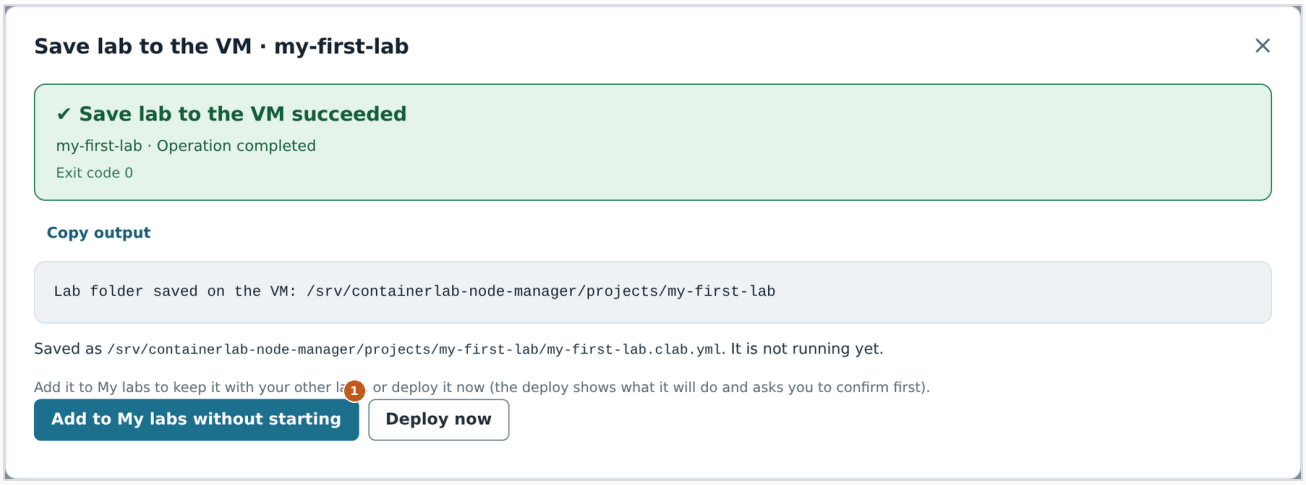


Figure B.3b — Saved on the VM; Deploy now starts it.

B4 Bring the link up

Action Wait for both devices to read **Ready** on the **Devices** tab — about half a minute on this VM; up to a few minutes on a slower one. Open a CLI on `r1` and run:

```
r1>enable
r1#configure terminal
r1(config)#ip routing
r1(config)#interface Ethernet1
r1(config-if-Et1)#no switchport
r1(config-if-Et1)#ip address 10.0.0.1/30
r1(config-if-Et1)#end
r1#write memory
```

`no switchport` matters here: without it, EOS keeps `Ethernet1` as a switched port and refuses the `ip address` that follows. On `r2`, run the same commands with `ip address 10.0.0.2/30`. From `r1`, run `ping 10.0.0.2`.

Expected result The ping succeeds.

```
Last login: Tue Oct 27 04:10:53 2026 from 172.20.20.1
r1>ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 72(100) bytes of data.
80 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.096 ms
80 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.007 ms
80 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.006 ms
80 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.005 ms
80 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.008 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.005/0.024/0.096/0.035 ms, ipg/ewma 0.042/0.059 ms
r1>
```

Figure B.4 — Link addressed on both routers; `r1` pings `r2`.

B5 Create your own repository on GitHub

Where to run it On your computer.

Action On GitHub, choose **New repository**. This is created under the same GitHub account the lab VM is signed in to — `ArchRuger` in this guide's example, your own account for you. Name it

(in this guide `campaign2-my-network-labs`; use your own name wherever you see it), set it **Private**, tick **Add a README file**, and click **Create repository**. From a terminal with GitHub's command-line tool installed, the same repository can be created with:

```
$ gh repo create campaign2-my-network-labs --private --add-readme
```

Copy its **Code → HTTPS** address for the next step.

Expected result The repository exists, private, on branch `main`, with a `README.md`.

B6 Connect the repository and make the first save

Action In the manager, open `my-first-lab` and click the save chip (**Not saved yet**). The panel may already propose a repository (the one you used last, for example in Scenario A) — do not take it. Click **Connect by URL…**; in the drawer **Where should my-first-lab save?** paste `https://github.com/ArchRuger/campaign2-my-network-labs.git` (your account) into **Repository address (HTTPS)**, leave **Folder** at `my-first-lab`, and click **Save here**.

Expected result The manager copies your repository onto the lab VM, connects it and makes the lab's first save — about fifteen seconds. The chip reads **1 save to upload**; its panel, **Not uploaded yet**, says "This is the first save here: 2 devices, the topology and the map." with "To: campaign2-my-network-labs › my-first-lab". **See changes** lists `Topology file my-first-lab.clab.yml added`, `Map my-first-lab.clab.yml.annotations.json added`, `r1 r1.cfg added` and `r2 r2.cfg added`. Click **Upload**.

Expected result The chip reads **Saved just now**. On GitHub, `my-first-lab/latest/` holds `manifest.json`, `my-first-lab.clab.yml`, `my-first-lab.clab.yml.annotations.json`, `r1.cfg`, `r1.eoscfg`, `r2.cfg` and `r2.eoscfg` — the lab definition and the device configurations together.

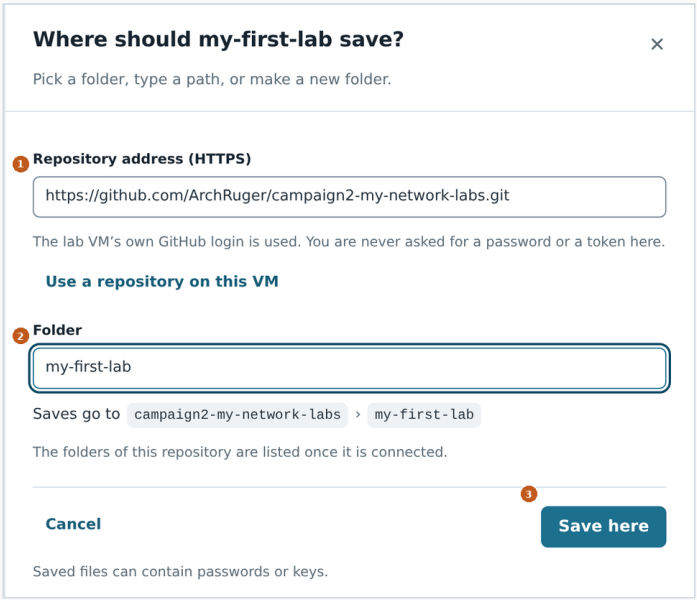


Figure B.6a — Connecting `campaign2-my-network-labs` with folder `my-first-lab`.

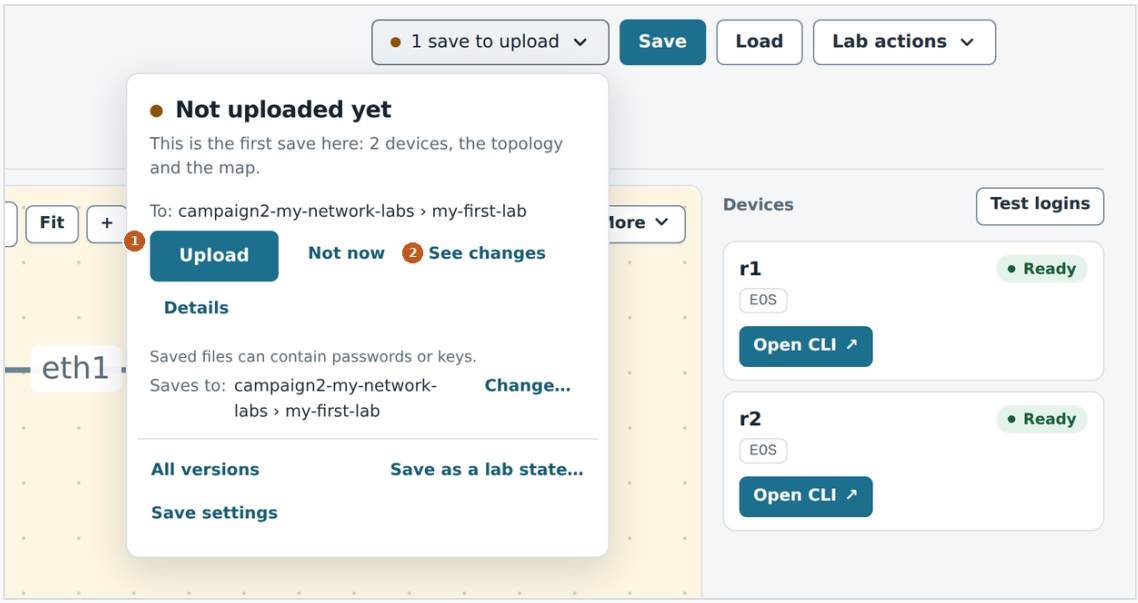


Figure B.6b — The first save, waiting for Upload.

B7 Check GitHub

Where to run it On your computer.

Action Open your repository on GitHub. Back in the manager, the same tree is one click away: click the chip, **All versions**, then **Browse the repository...**.

Expected result GitHub shows the `my-first-lab` folder, with `latest/` inside it, next to your `README.md`. The manager's **Browse the repository** drawer shows exactly the same: "Every folder of campaign2-my-network-labs", with **This lab saves here** beside `my-first-lab`, and below the tree the files and folders of the top level.

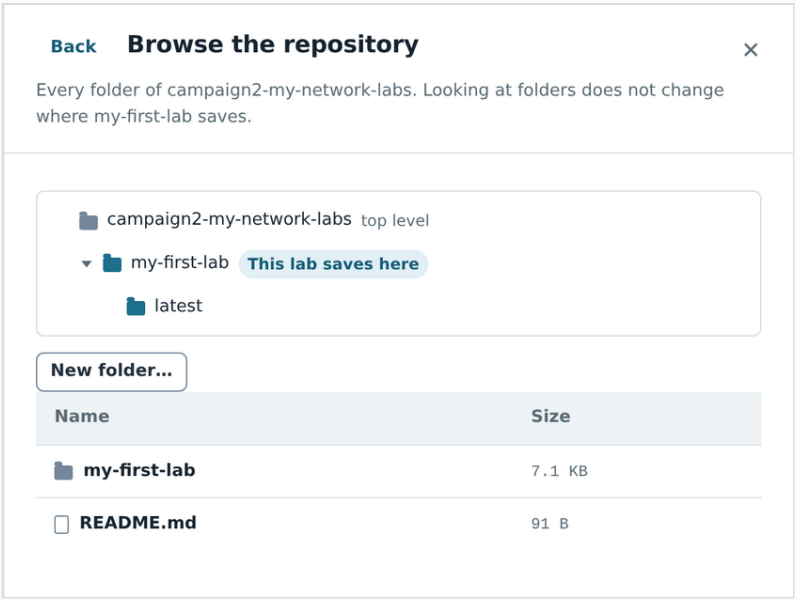


Figure B.7 — Browsing the repository from the save chip.

B8 Change, save, and checkpoint again

Action On `r2`'s CLI, add a `Loopback0` addressed `10.255.0.2/32`, the same way as in Scenario A, then `write memory`. Click **Save**.

Expected result The sentence reads "1 device changed since your last save: r2. 4 lines added, 0 removed." — `r1` did not change. Click **Upload**; the chip reads **Saved just now**.

Action Click the chip, type `link-up` into the name field and press Enter, tick **Keep as a checkpoint**, and click **Upload** when the panel says "Checkpoint link-up kept. It is not uploaded yet."

Expected result Click the chip, then **All versions**: **Your saves** lists **link-up** and **First save**, and **Checkpoints** lists `link-up`. On GitHub, `my-first-lab/latest/r2.cfg` contains the new `Loopback0` and `my-first-lab/checkpoints/link-up/` holds the same seven files; there is no nested `latest/latest`.



*Figure B.8a — Naming the save `link-up`; the box **Keep as a checkpoint** is under the name.*

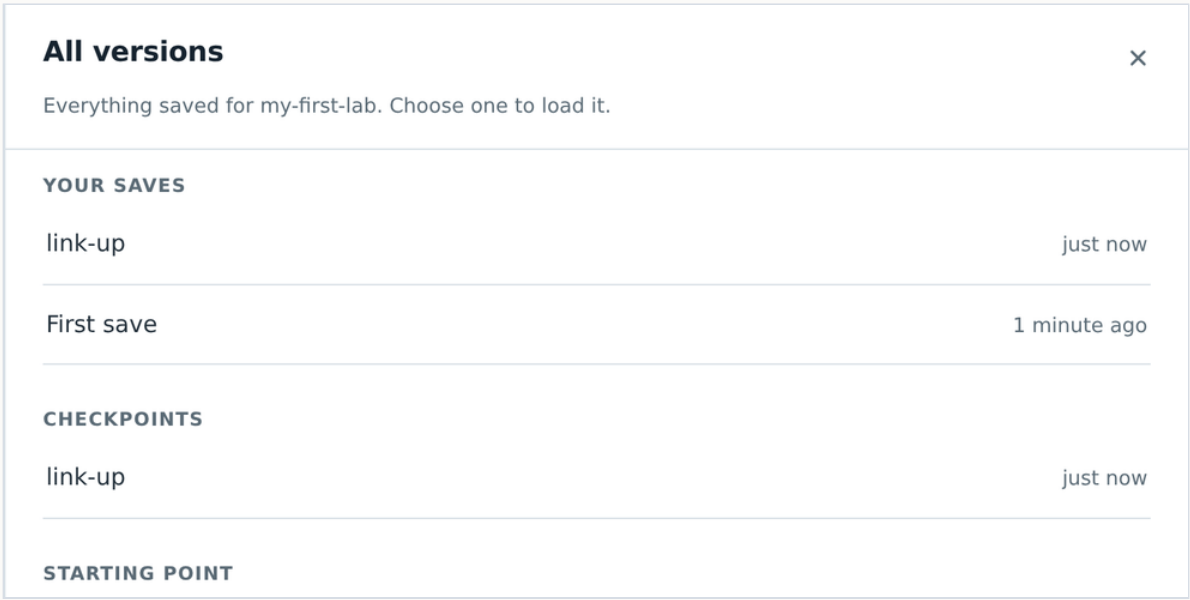


Figure B.8b — All versions: your saves and the checkpoint.

B9 Prove it: destroy, remove, and rebuild from Git

Action From **Lab actions ▾**, choose **Destroy lab…**. The review, titled **Destroy my-first-lab?**, warns the same way as in Scenario A; confirm with **Destroy lab**.

Expected result The lab header reads **Stopped** — the devices are gone.

Action From **Lab actions ▾**, choose **Remove from this manager…** and confirm with **Remove lab**. Its checkbox **Don't offer this lab for import again** can stay ticked or not — either is fine here. This step only removes the lab from this manager; it never touches your repository or the topology file saved in B3.

Where to run it On the lab VM, as your ordinary account. Clone your repository into a trusted lab folder, since the manager's deploy browser only shows folders it trusts:

```
$ git clone https://github.com/ArchRuger/campaign2-my-network-labs.git /srv/containerlab-node-manager/projects/campaign2-my-network-labs
```

Expected result `Cloning into '/srv/containerlab-node-manager/projects/campaign2-my-network-labs'...`

If not If the lab VM refuses to write into that folder, ask your instructor to add your VM account to the lab-folder group.

Action Back in the manager, from Home click **Choose a file on the lab VM…**, open `/srv/containerlab-node-manager/projects` → `campaign2-my-network-labs` → `my-first-lab` → `latest` (the folders inside the clone) → click ◇ **my-first-lab.clab.yml**. In the **Topology file** dialog, check that **File location on the VM** reads `/srv/containerlab-node-manager/projects/campaign2-my-network-labs/my-first-lab/latest/my-first-lab.clab.yml` — it must contain your repository's name. The original `my-first-lab` folder is still listed at the top level and looks the same; picking that one instead would deploy the old copy, not the one you just rebuilt from Git. Click **Deploy lab**, then confirm the **Start my-first-lab?** review with **Start lab**.

Expected result Both devices read **Ready** and the Topology tab shows the same map as before — the map came from Git along with the topology. The devices themselves are brand new: they have no addresses, so a ping between them fails, and the chip reads **Not saved yet**.

Action Connect the lab to your repository again: click the chip, **Choose another place**, pick `campaign2-my-network-labs` in the **Repository** list. The folder `my-first-lab` is what you saved before, so the manager asks: "This folder holds the state “My-first-lab”." — it is your earlier work, whose lab you removed. Click **Replace it**: the next save of the rebuilt lab continues in that folder, and the older contents stay in the Git history.

Expected result The manager makes a save of the fresh devices at once ("2 devices changed since your last save: r1 and r2. …"). Do not upload it: click **Not now**.

Action Click **Load**. **Your saves** lists the fresh save and your checkpoint **link-up**. Click **link-up**; the confirmation **Load link-up?** lists both devices; click the red **Load**.

Expected result Within about ten seconds the chip reads **Running link-up**. On `r1`'s CLI, `ping 10.0.0.2` succeeds, and `show ip interface brief` on `r2` lists `Loopback0 10.255.0.2/32` — the lab is rebuilt entirely from what Git kept.

Action Click **Save**: the sentence names what this save changes and adds "This upload also sends 1 other save", the fresh one you kept back. Click **Upload** so that your repository holds the rebuilt lab as its latest save.

Lab VM terminal (rendered transcript, not a live terminal)

```
$ git clone https://github.com/ArchRuger/campaign2-my-network-labs.git /srv/containerlab-node-manager/projects/campaign2-my-network-labs
Cloning into '/srv/containerlab-node-manager/projects/campaign2-my-network-labs'...
```

Figure B.9a — Cloning the repository into the trusted lab folder (a transcript, not a live terminal).



Figure B.9b — The Topology file dialog: the path runs through your repository's folder.

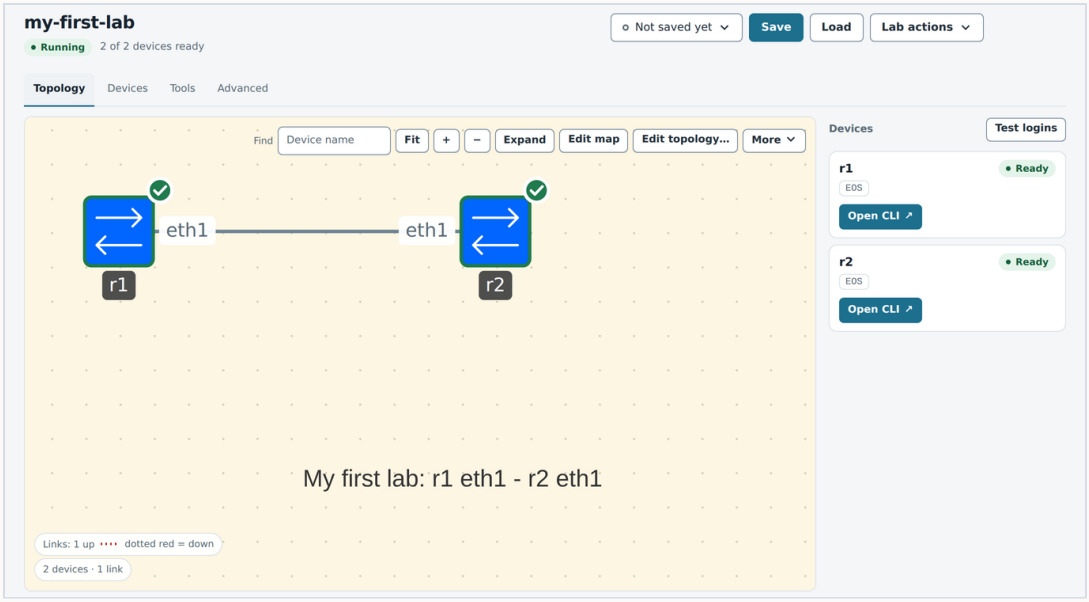


Figure B.9c — The rebuilt lab: the saved map, devices Ready.

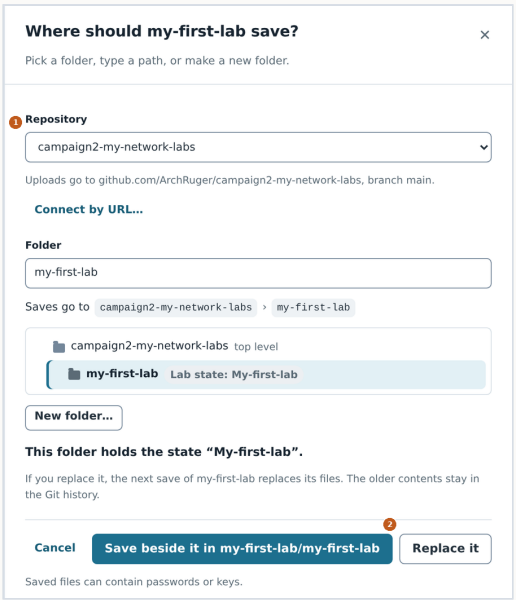


Figure B.9d — Choosing the repository again: the folder holds your earlier work.

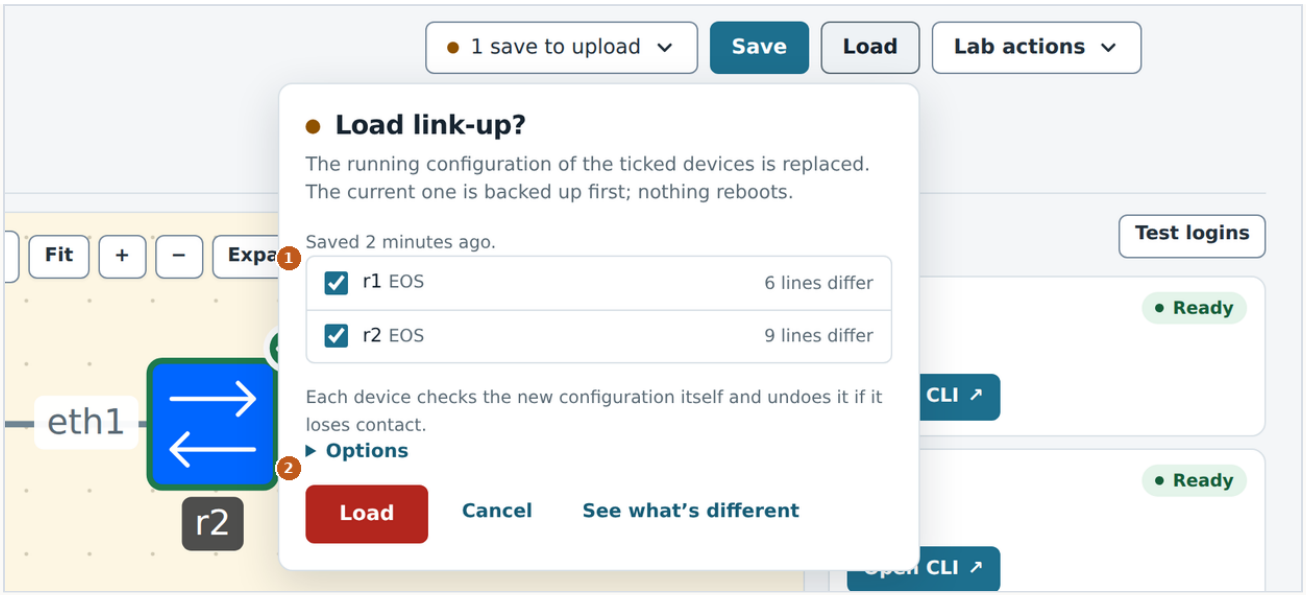


Figure B.9e — Load link-up?, before anything changes.

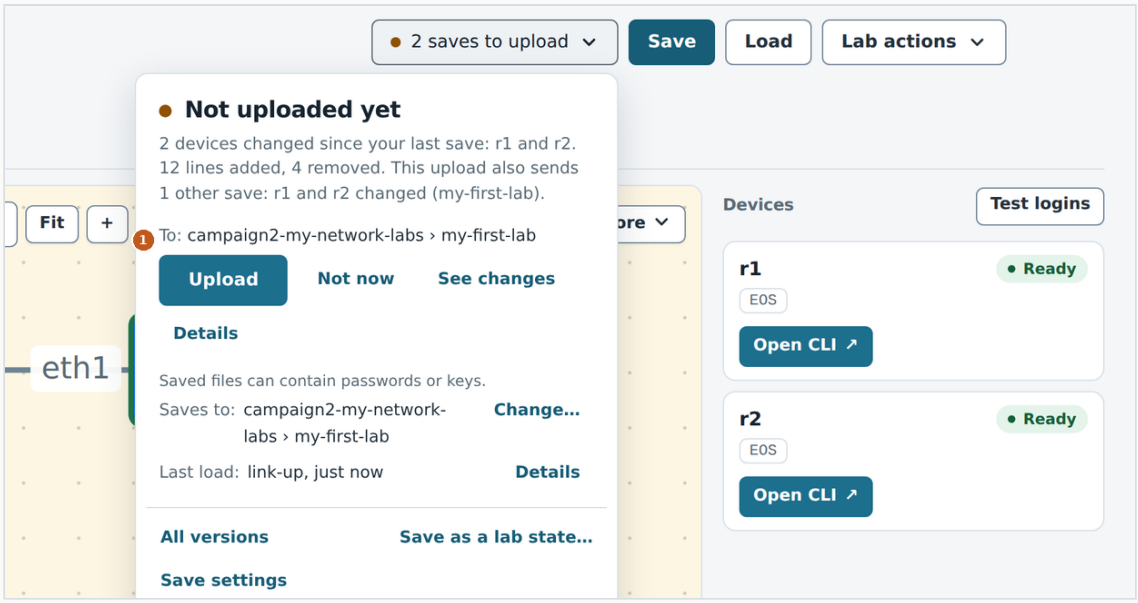


Figure B.9f — The final save, with the kept-back save in the same upload.

Checklist

- [ ] `my-first-lab` drawn, imaged correctly, saved on the VM and deployed
- [ ] Link addressed and pinging
- [ ] `campaign2-my-network-labs` created on GitHub, private, with a first commit
- [ ] Repository connected, first save uploaded (topology, map and configurations)
- [ ] A second change saved, and checkpoint `link-up` created
- [ ] Lab destroyed and removed, then redeployed from the folder inside the cloned repository
- [ ] Repository chosen again, **link-up** loaded; the link pings and r2's loopback is back

Reference

What is saved where

What it holds	Where it lives	Created by	Survives a device restart?	Survives a Destroy lab?
A drawn but unsaved topology	This browser's storage	The lab builder, as you edit	Not applicable — no lab is deployed yet	Not applicable
The topology file and its map	A lab folder on the lab VM	**Save to the VM…** / **Deploy lab**	Yes	Yes — Destroy removes the running devices and the generated lab folder, not the topology file
A device's running configuration	The device itself	Commands you type; `write memory` on EOS	Only if written to startup	No — Destroy removes the containers
A save waiting for upload (chip **1 save to upload**)	The repository's copy on the lab VM	**Save**, then **Not now**	Yes	Yes
An uploaded save (`latest/`): the topology, the map and every device's configuration	Your online repository	**Upload**	Yes	Yes
A checkpoint	Your online repository (`checkpoints/<name>/`; on the lab VM until uploaded)	**Keep as a checkpoint**	Yes	Yes
The result of **Load**	The running devices	**Load**, confirmed with the red **Load**	Yes — on EOS the manager runs `write memory` for you once the change is confirmed	No — Destroy removes the containers; redeploy and load again

Stop, Destroy, Remove

Stop devices keeps the containers so you can start them again quickly; nothing configured is lost, but it is also not saved anywhere by stopping. **Destroy lab…** removes the running devices (and, on this manager, the generated lab folder with them) — configuration changes you never saved are lost, but your saves, lab states and backups all remain, since they were never stored on the devices; its review offers **Save first**. **Remove from this manager…** only forgets the lab in this manager; it never deletes anything on the lab VM or in your repository, and a lab it removed can always be added back by deploying its topology file again. A lab with a save still waiting for upload cannot be removed until you upload it or keep it on the VM only (**Details** on that save, then **Keep snapshot only**).

Troubleshooting

Symptom	Next step	Ask your instructor if…
A device stays on **Starting** for several minutes	Wait — cEOS and similar network operating systems take up to a couple of minutes to boot and answer a login	It never reaches **Ready**
A device shows **Needs credentials**	Ask your instructor for the login this lab expects, enter it, then click **Test login now** in the device panel	You do not know the intended login for this lab
The chip reads **1 save to upload** or **Upload failed**	Click the chip and press **Upload** (or **Try again**); your save is safe on the lab VM meanwhile	The upload keeps failing after several tries
The chip reads **Can’t save**, or **Save** is greyed out	Read the sentence in the chip panel and the reason beside **Save**; "A backup or lab operation is running. Save is available when it finishes." clears by itself	The reason is not one of these
Load or **Deploy** answers "Wait for the current lab operation to finish."	Another operation is running on the manager (it may belong to another lab); wait for it, then click again	It never finishes
A row in **Load** reads **View only**, **Not in this lab**, or **Not reachable**	Read the reason under it; a state fits a lab with the same name and device names, and the lab must be running	The reason is not one of these
The manager cannot reach or push to the repository	Check you connected your own copy of the repository, not a read-only one	The message names a permission or repository problem you cannot resolve
Deploy fails naming an image the VM does not have	Open the lab with **Edit topology…**, correct the device's **Image** field, save, and deploy again	The image you were told to use still is not found

Next session in 60 seconds

- Open the manager, open your lab from Home, click **Start lab** if needed, and wait for **Ready**.
- Look at the save chip: **1 save to upload** means something waits for **Upload**.
- Pick up where you left off, or **Load** a saved state to jump to a specific point.
- End the session with **Save** and **Upload**.